Improving CNN Performance Using Data Augmentation: An Empirical Study

Student ID : 24144276

1. Introduction

Convolutional Neural Networks (CNNs) have transformed image classification by enabling the automatic extraction of hierarchical features from raw image data. Traditional machine learning approaches depend on manually engineered features, which require domain expertise and may not generalize across datasets. In contrast, CNNs learn features automatically through stacked convolutional layers, pooling layers, and non-linear activations, capturing both low-level features such as edges and textures and high-level semantic features such as object shapes and patterns.

Despite their strengths, CNNs remain susceptible to overfitting, especially when trained on limited datasets. Overfitting arises when the model learns patterns specific to the training set rather than generalizable features, reducing its effectiveness in real-world scenarios with varying data distributions.

Data augmentation is a widely used technique to increase the effective size and diversity of training datasets. Applying transformations such as rotations, flips, random cropping, zooming, brightness adjustment, and color jittering introduces variability that encourages models to learn robust features. This approach is especially valuable when collecting additional labeled data is costly or impractical.

The objective of this study is to empirically investigate the effect of data augmentation on CNN performance. This study aims to empirically assess the effect of data augmentation on CNN performance. We compare models trained with and without augmentation using the CIFAR-10 dataset. In addition to reporting validation accuracy and loss, we discuss factors influencing whether augmentation improves performance in specific experimental contexts. binning convolution, pooling, and dense layers to extract hierarchical features for classification. Convolutional layers use filters to generate feature maps, capturing visual patterns and preserving spatial relationships. **Pooling layers summarize features by reducing their spatial resolution, thereby reducing computation and helping the model focus on the most prominent information.**

2 Fully connected layers flatten outputs to classify using learned weights. Fully connected layers flatten outputs for classification using learned weights. Nonlinear activations, such as ReLU, allow CNNs to model complex data patterns. The combination of these layers enables automatic feature extraction for visual tasks.ns, such as rotations, flips, zooming, cropping, and color changes. Random rotation: Rotating images by a small angle (e.g., ±15 degrees) helps the model recognize objects regardless of orientation.

- Horizontal flipping: Flipping images horizontally helps the model learn invariance to mirrored patterns.
- Zooming in/out: Randomly resizing images ensures scale-invariance, allowing the model to recognize objects of varying sizes.
- Random cropping: Extracting random subregions simulates shifts and translations.
- Color jittering and brightness adjustment: Slight variations in color and intensity help the model generalize to different lighting conditions.

Data augmentation serves multiple purposes:

1. Increasing dataset size: Even small datasets can be expanded many times.
2. Reducing overfitting: By preventing the model from memorizing specific examples, augmentation encourages generalization.
3. Improving robustness: Models learn to recognize patterns under diverse conditions, enhancing performance on real-world data.

Research indicates that data augmentation can enhance CNN performance, although the extent of improvement depends on the model architecture, dataset size, and augmentation strategy.

3. Methodology

3.1 Dataset

We used the CIFAR-10 dataset, a widely adopted benchmark in image classification research. CIFAR-10 contains 60,000 32x32 color images distributed across 10 different categories of everyday objects. **The training set contains 50,000 images, and the remaining 10,000 images are reserved for testing to evaluate model performance.**CIFAR-10 was selected because it balances rapid experimentation with sufficient complexity to enable meaningful evaluation of CNN generalization and the effects of augmentation.

3.2 Experimental Setup

We trained two CNN models with identical architectures and hyperparameters:

1. Model 1 (Baseline): Trained solely on the original training images without augmentation.
2. Model 2 (Augmented): Trained on augmented data using random rotations (±15 degrees), horizontal flips, and zooming (0.9–1.1×).

Both models were trained for 5 epochs using Adam with a learning rate of 0.001 and a batch size of 32. The loss function used was categorical cross-entropy. Using identical training conditions ensures that differences in performance are attributable solely to data augmentation.

3.3 CNN Architecture

The CNN architecture consisted of:

- Input layer: Accepts 32x32x3 images.
- Convolutional Layer 1: 32 filters, kernel size 3x3, ReLU activation.
- Max Pooling Layer 1: Pool size 2x2.
- Convolutional Layer 2: 64 filters, kernel size 3x3, ReLU activation.
- Max Pooling Layer 2: Pool size 2x2.
- Flatten layer: Converts 2D feature maps into a 1D vector.
- Fully Connected Layer: 128 neurons with ReLU activation.
- Output Layer: 10 neurons with softmax activation for classification.

This architecture is intentionally simple to isolate the effect of augmentation without introducing confounding complexity from deeper networks.

3.4 Evaluation Metrics

Performance was evaluated using validation accuracy and validation loss. Accuracy reflects the proportion of correctly classified images, while loss quantifies the divergence between predicted and true class distributions.

Additionally, visual inspection of augmented images was performed to confirm that transformations were reasonable and label-preserving.

4. Results

4.1 Visualizing Augmented Images

Inserted the augmented images, which consist of rotated, flipped, and zoomed versions of the originals. Visual inspection confirmed that these transformations preserved the original labels and increased dataset diversity.

4.2 Validation Accuracy

The baseline model achieved a final validation accuracy of 67.58%, whereas the augmented model reached 64.95%. Contrary to expectations, the baseline consistently outperformed the augmented model across all epochs, highlighting the complexities of augmentation's impact on CNN performance.

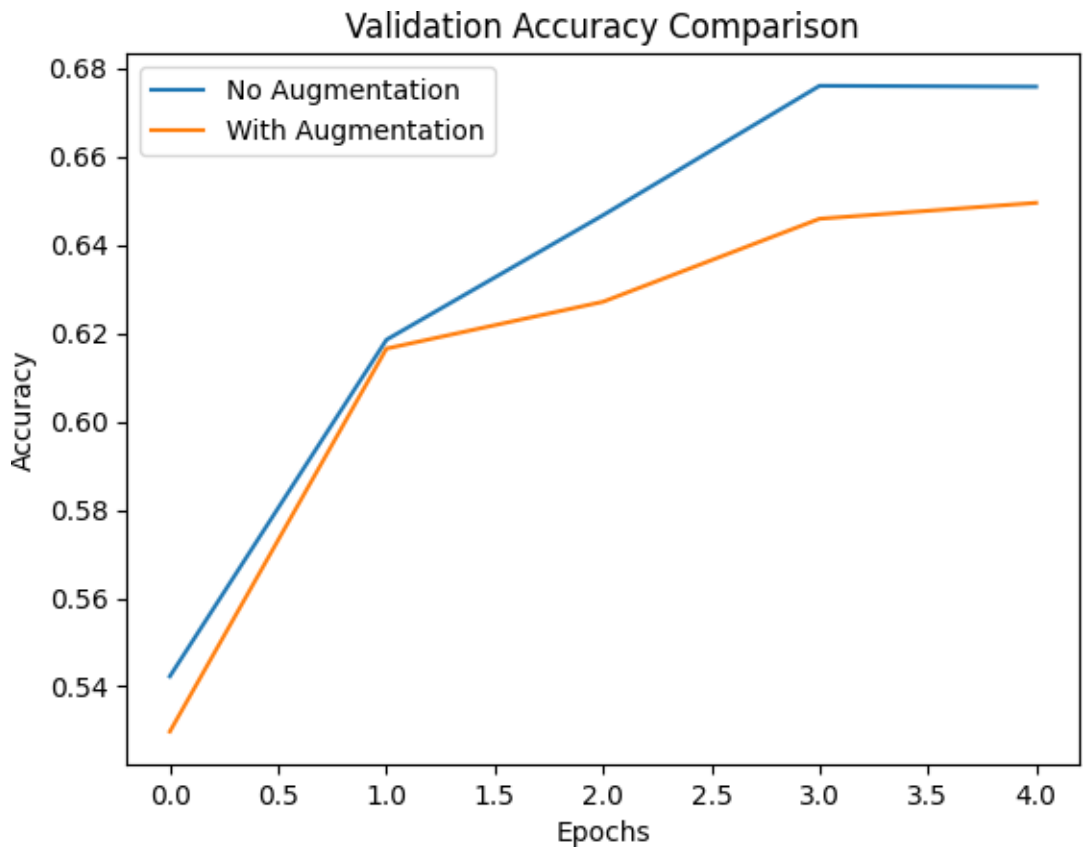

Figure 1: Validation accuracy of CNN models trained with and without data augmentation over 5 epochs.

4.3 Validation Loss

Validation loss decreased for both models. The baseline model achieved a lower final loss (0.9484) than the augmented model (0.9927), indicating more confident predictions.

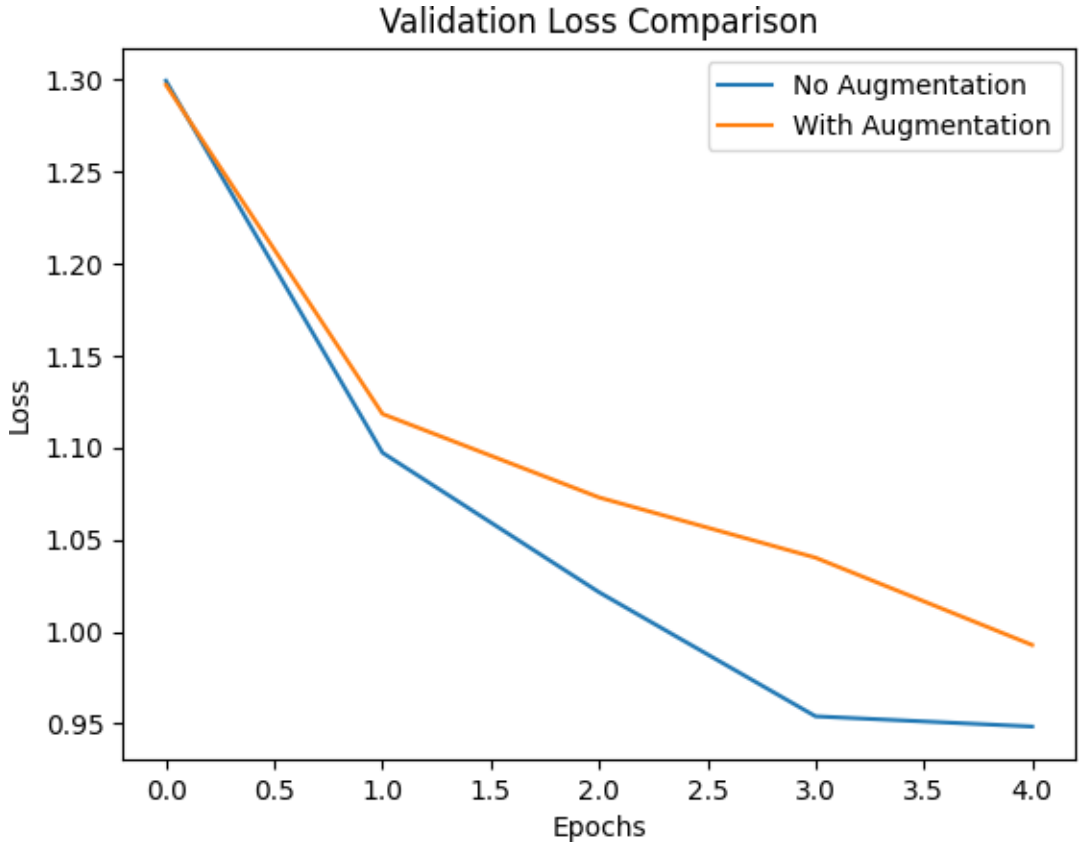

Figure 2: Validation loss of CNN models trained with and without data augmentation over 5 epochs.

4.4 Final Quantitative Results

Without Augmentation	67.58%	0.9484
With Augmentation	64.95%	0.9927

5. Discussion

The baseline model outperforming the augmented model appears counterintuitive, as data augmentation is generally expected to enhance performance. Several factors likely contributed to this outcome:

1. Limited Training Duration: Augmented datasets increase the complexity of training. Short training schedules may prevent models from fully leveraging additional data, slowing convergence.
2. Model Capacity: The simple CNN architecture may lack sufficient depth and parameters to effectively exploit augmented diversity. More complex models generally benefit more from augmentation.
3. Augmentation Strategy: The applied transformations may have been too mild or insufficiently varied to significantly impact learning. Aggressive augmentation strategies might yield different outcomes.
4. Training Dynamics: Augmentation introduces additional variability that the model must learn to accommodate, which can temporarily reduce performance if the model is undertrained.

This experiment demonstrates that data augmentation effectiveness depends on appropriately tuned training schedules, adequate model capacity, and suitable augmentation parameters.

6. Conclusion

This study shows that with brief training and simple CNN architectures, augmentation does not always improve accuracy. However, augmentation remains essential for developing robust, generalized models with greater capacity or extended training.

Practical recommendations:

- Carefully select augmentation types and intensities based on the dataset's characteristics.
- Ensure sufficient training duration for models to learn from augmented data.
- Combine augmentation with architectures that can leverage increased data diversity.
- Use visual inspection to verify label-preserving transformations.

Future work should explore more complex CNNs, longer training regimes, and a wider variety of augmentation strategies to fully understand and utilize the benefits of data augmentation.

7. Code and Reproducibility

The full implementation of this study is available in the GitHub repository. The code includes:

- Data preprocessing and augmentation using TensorFlow/Keras.
- Model definition and compilation.
- Training loops with configurable hyperparameters.
- Visualization of accuracy, loss, and augmented images.

This approach ensures transparency and enables other researchers to replicate the study or explore modifications.

8. Accessibility and Clarity

This tutorial emphasizes clear communication for accessibility:

- Step-by-step explanations for all methodology choices.
- Annotated figures showing examples of augmented images and training curves.
- Use of standard terminology with definitions for beginners.
- Structured headings and subheadings to improve readability.

These measures ensure that readers with varying levels of expertise can understand and apply the tutorial effectively.

9. Reference

A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

TensorFlow Documentation, "Convolutional Neural Networks (CNN)," [Online]. Available: https://www.tensorflow.org/tutorials/images/cnn

My GitHub Link: https://github.com/Pampapathi25/CNN_Performance